Data Mining: Recommender Systems

Fall 2026 (7761)

Dept. of Computer Science and Engineering

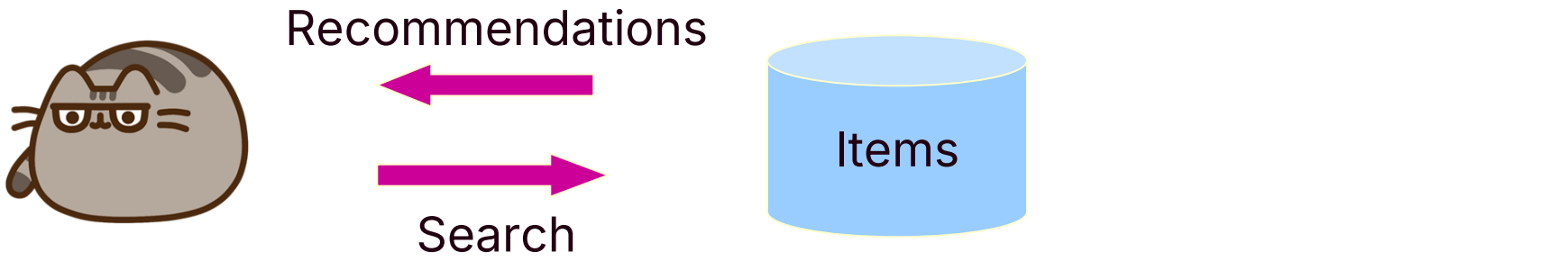
Instructor: Ting-Wei Lin

What Is a Recommender System?

- A recommender system helps users discover items they may prefer or find useful.
  - It learns user preferences from feedback, interactions, and available user or item information.
- Common recommendation tasks include:
  - Rating prediction: Estimate a user's rating for an unrated item.
  - Interaction prediction: Estimate whether a user will click, watch, or purchase an item.
  - Top-N recommendation: Rank candidate items and recommend the N most relevant items to a user.

Search vs. Recommendation

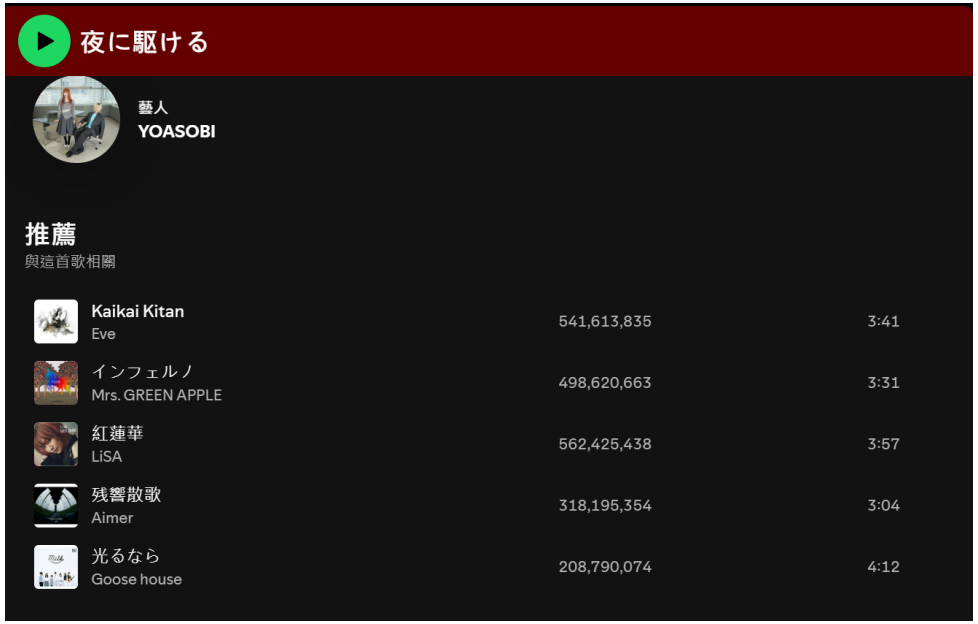
- Search: matches items to a query.
  - An explicit query.
  - Find items relevant to the query.
- Recommendation: matches items to a user.
  - User preferences, interactions, and context.
  - Discover items relevant to the user.

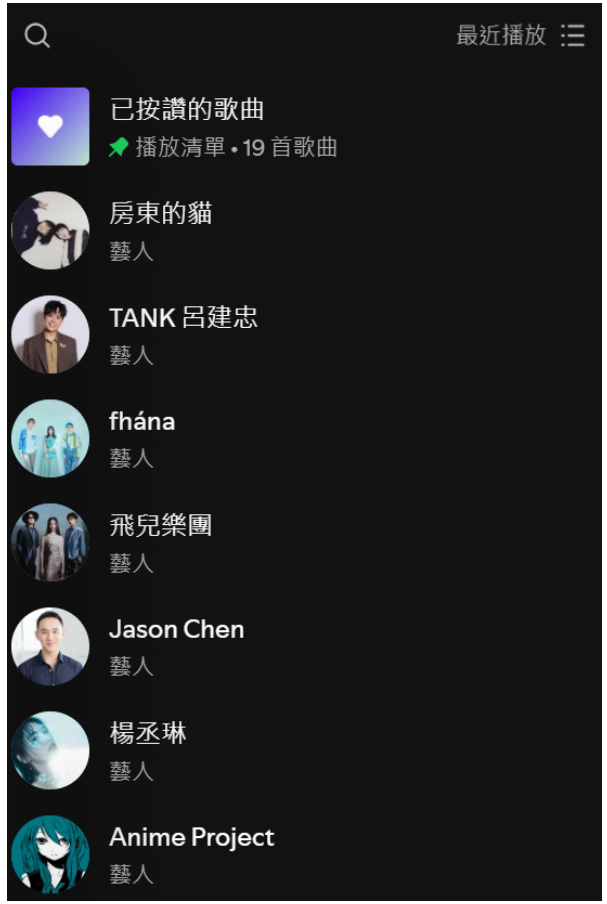


Products, websites, blogs, news items, ...

Why recommendation?

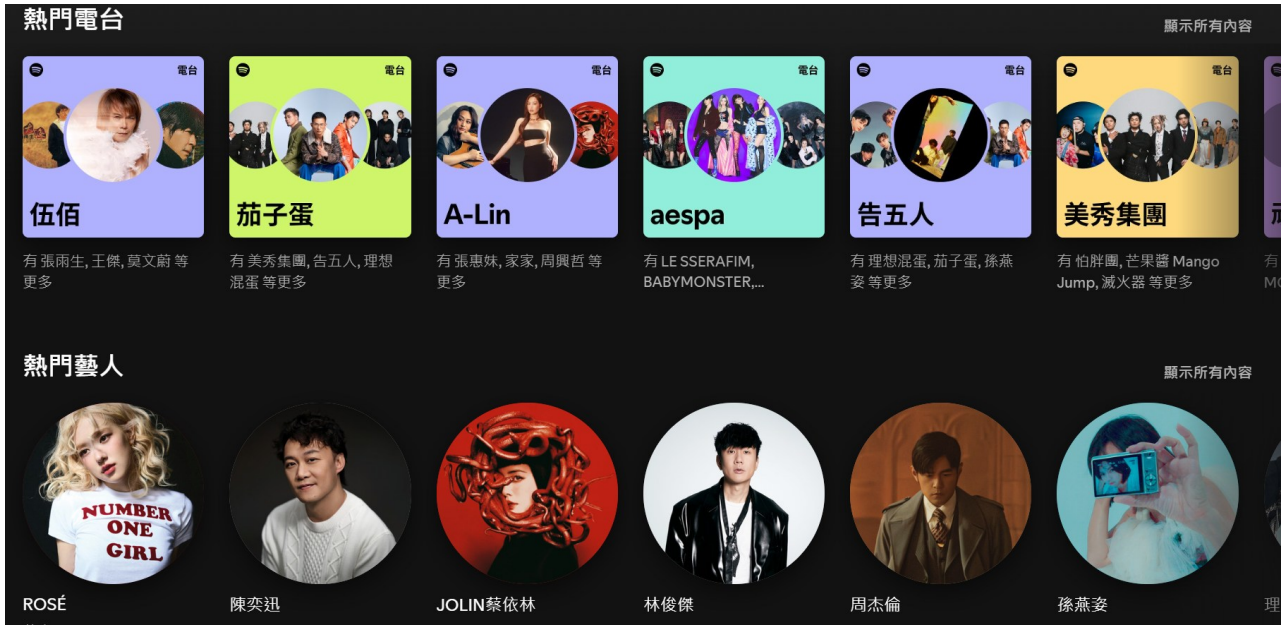
- To help users find relevant content efficiently.
  - Recommend music from familiar artists.
  - Discover similar songs.
  - Continue a playlist with relevant songs.





Popularity-Based Recommendation

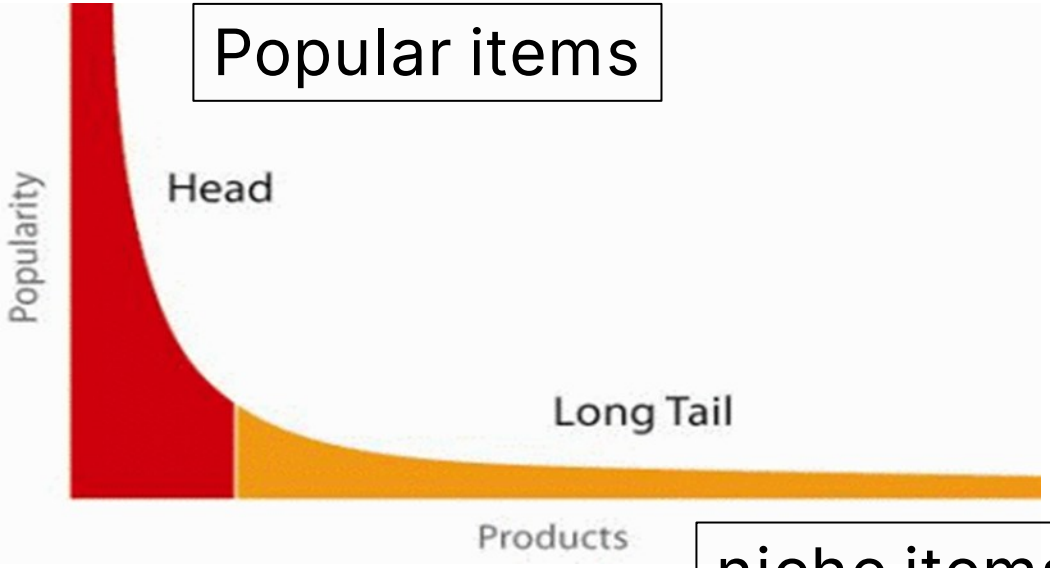
- Recommend items that are popular across users.
  - Rank items by interaction counts, such as plays or purchases.
  - Provide a simple baseline for evaluating personalized methods.
  - Can serve users with little or no interaction history.

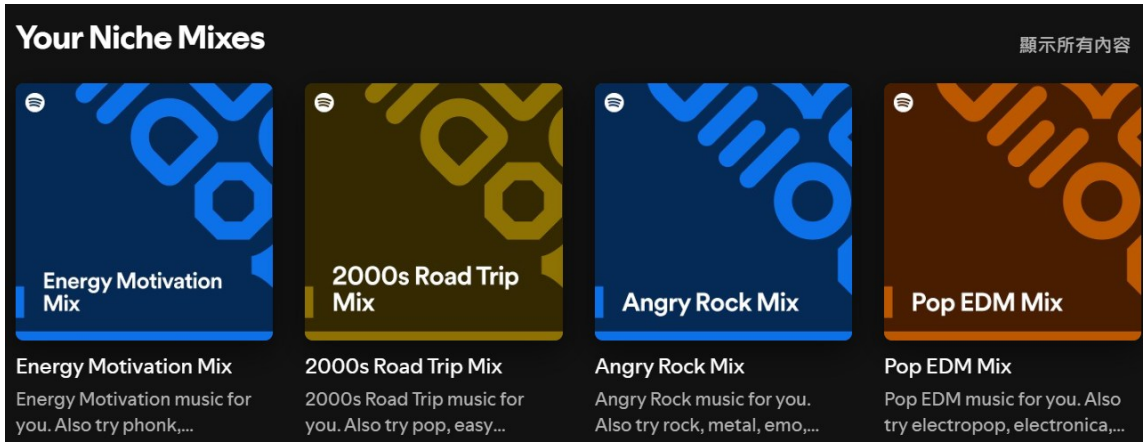


A simple baseline.

Why recommendation?

- To help users discover content beyond what they already know.
  - Recommend unfamiliar items that match their interests.
  - Help users discover less popular or niche items in the long tail.



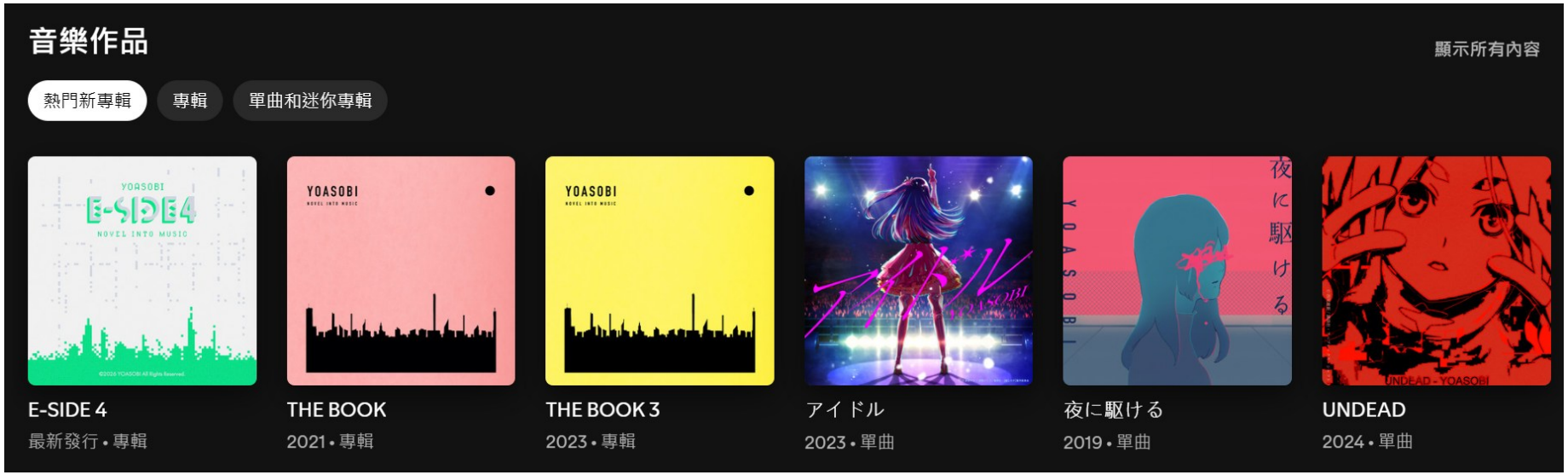


Balancing Relevance and Diversity

- Balance: provide varied recommendations while maintaining relevance.
  - Relevance: recommend items that match the user’s interests or needs.
  - Diversity: include a variety of items within the recommendation list and avoid overly repetitive recommendations.
- Example: A user enjoys rock music
  - Less diverse: Songs from a single rock band
  - More diverse: Songs from several rock artists and subgenres

Why recommendation?

- To help users explore related content
  - Help users discover other works by an artist they enjoy.
  - Connect users to relevant albums, songs, and artists.
  - Reduce the time and effort needed to decide what to consume.



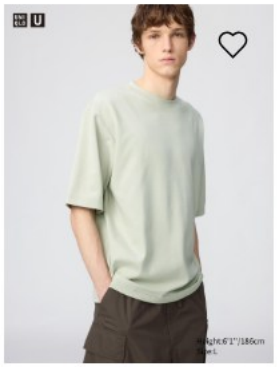
穿搭單品 Complementary Item Recommendation



UNIQLO_TW

疊穿趨勢正流行，選對「褲裝輪廓」更是風格關鍵！✨ 透過輪廓的轉變與探討穿搭方式的可能，成為了「建立穿搭品味與風格」的存在，本季所強調的「新色彩・新輪廓」，也以更自由的前提，為每個人勾勒出屬於春夏的全新想像。 與新銳演員章廣辰 & 新生代女星 詹子萱 一起迎接一個自由的春夏！🪄 #UNIQLOTAIWAN #LifeWear #章廣辰 #詹子萱 #新色彩新輪廓

2026/04/30更新



男裝/男女適穿

AIRism棉質寬版圓領T恤(五分袖) 475355

顏色：482295 / 02 LIGHT GRAY

NT$490

查看商品詳情



男裝/男女適穿

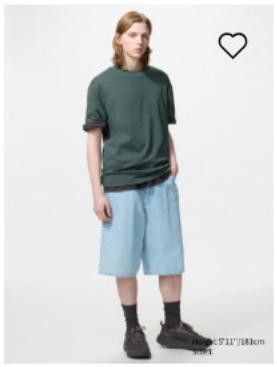
寬版襯衫(短袖)

顏色：06 GRAY

NT$990

查看商品詳情

Recommend items that work well together, such as a shirt, shorts, and shoes.



男裝/男女適穿

寬版Jorts牛仔短褲

顏色：08 DARK GRAY

NT$790

查看商品詳情

Why recommendation?

- To personalize recommendations based on user feedback.
- User preferences can be inferred from interaction history:
  - Clicks
  - Listening or watch time
  - Purchases
  - Skips



Interaction history →
User profile →
Personalized recommendations

- Different users may receive different recommendations

Explicit vs Implicit Feedback

- Explicit feedback:
  - Directly expressed preferences, often sparse.
  - Ratings, likes/dislikes, reviews.
- Implicit feedback:
  - Preferences inferred from behavior, often abundant but ambiguous.
  - Clicks, watch time, purchases, skips.
- An interaction does not always indicate liking
- No interaction does not necessarily indicate dislike.

Why Recommendation?

- The same user may prefer different items in different situations: context matters
  - Time of day
  - Current activity
  - Location or device
- Recommend items that fit both the user and the current context
  - The same listener may prefer calm music while studying and energetic music while exercising.

Why Recommendation?

- Value for different stakeholders:
  - Users: Discover useful and enjoyable items.
  - Platforms: Improve engagement, retention, and conversion.
  - Creators or sellers: Reach relevant audiences and gain exposure.
  - These objectives may not always align.
- Maximizing short-term clicks may not maximize long-term user satisfaction

Recommendation Systems

- For each user c, recommend the item s with the highest predicted utility:

$$s_c^* = \underset{s \in S_c}{arg\max}\, \hat{u}(s, c)$$

Song	Predicted Score
Song A	0.9
Song B	0.3
Song C	0.8

For user c: $S_c^* = song\ A$

- Notation
  - $c \in C$: a user.
  - $s \in S_c$: a candidate item for user c.
  - $\hat{u}(s, c)$: the predicted preference score of user c for item s.
- For Top-N recommendation, rank candidate items by predicted scores and return the N highest-ranked items.

Main Recommendation Approaches

① Content-based filtering: match the user's preferences.

② Collaborative filtering: user–item interactions across users.

- Memory-based CF
  - User-based CF: Use feedback from similar users.
  - Item-based CF: Use item similarities inferred from interaction patterns.
- Model-based CF
  - Learn a predictive model from user–item interactions, such as matrix factorization

③ Hybrid: Combine multiple strategies, such as content-based and collaborative filtering.

CBF vs. CF

	Content-Based Filtering	Collaborative Filtering
Data Source	Item content/features and the user's feedback	User-item interactions
Cold Start	Can recommend new items if features are available, but needs information about new users	Difficult for new users and new items without interactions
Diversity	May overspecialize in familiar item features	Can discover items beyond content similarity
Scalability	Depends on feature extraction and the scoring method	Neighborhood similarity computation can be costly
Explainability	Often easier with explicit features	Often less interpretable
Personalization	Match item features to an individual user's preferences	Combine the user's history with interaction patterns across users

Content-based Recommendation

Similarity-Based Recommendation

Classification-Based Recommendation

Content-based Recommendation

- Key assumption: Item content features can help predict the user's preferences.
  - Represent each item s by a content feature vector $\boldsymbol{x}_s$.
  - Learn a preference model f_c from user c's past feedback.
  - Predict a preference score for each candidate item:

$$\hat{u}(s, c) = f_c(\boldsymbol{x}_s)$$

  - Rank candidate items by their predicted scores.
- Common Approaches
  - Similarity-Based Recommendation
  - Classification-Based Recommendation

Similarity-Based Recommendation

- Procedure
  ① Represent items using content feature vectors.
  ② Build a user profile by aggregating features of liked items.
  ③ Compute similarity between the user profile and candidate items.

$$\hat{u}(s, c) = \mathrm{sim}(\boldsymbol{p}_c, \boldsymbol{x}_s)$$

    - $\boldsymbol{p}_c$: the user's preference vector
    - $\boldsymbol{x}_s$: the item's content feature vector

  ④ Recommend the N highest-scoring candidate items.

User Profiles

- Summarize the user’s interests, preferences, needs, and behavioral patterns.
- User profiles can be built from:
  - Explicit feedback: Ratings, questionnaires, likes/dislikes
  - Implicit feedback: Clicks, listening or watch time, purchases, browsing history
- For similarity-based content recommendation, the user profile and item vectors share the same content feature space.

Similarity and distance measures

- Cosine similarity, dot product, jaccard similarity : higher scores rank first
  - Dot product depends on vector magnitudes.
  - Cosine similarity compares directions.
  - Jaccard similarity is suitable for sets or binary features.
- Euclidean distance: smaller distances rank first

Example: Music Recommendation

- Music genre: Pop, Rock, R&B, Jazz, Classical, Reggae
  - User values represent genre preference weights
  - Song values represent genre characteristics.
  - $User\ U$: $\boldsymbol{p}_U = [10, 8, 0, 8, 0, 0]$
    - User strongly prefers: Pop, Rock, Jazz
  - $Song$ A: $\boldsymbol{x}_A = [9,7,0,6,0,0]$
    - $cosine(\boldsymbol{p}_U, \boldsymbol{x}_A) = 0.997195$
    - This song is highly similar to the user profile
  - $Song$ B: $\boldsymbol{x}_B = [1,0,10,0,0,6]$,
    - $cosine(\boldsymbol{p}_U, \boldsymbol{x}_B) = 0.056581$
- Song A ranks above Song B for this user.

Classification-Based Recommendation

- For a given user, treat recommendation as binary classification.
  - Input: Item content features.
  - Training labels: Explicit likes/dislikes or labels derived from ratings.
  - Output: A score or probability indicating whether the user will like the item.
- Rank candidate items by the predicted scores.

Classification-Based Recommendation

- Binary classification:

$$y_{cs} = \begin{cases} 1, & explicit\ like, \\ 0, & explicit\ dislike. \end{cases}$$

$$\hat{p}_{cs} = P(y_{cs} = 1 | x_s; \theta_c)$$

  - θ_c: parameters of the preference classifier for user c.
  - Use $\hat{p}_{cs}$ as the recommendation score.
- Other supervised formulations
  - Rating prediction: classification, regression
  - Watch-time prediction: regression

Unobserved items remain unlabeled; they are not automatically dislikes.

Limitations of Content-Based Recommendation

1. Limited Content Understanding
   - Difficult to extract high-level semantic features
   - Some user preferences are abstract or subjective
   - High-level qualities, such as a movie’s atmosphere or its emotional impact on a viewer, can be difficult to capture reliably with content features.

Limitations of Content-Based Recommendation

2. Insufficiently Discriminative Features
   - Feature representations may omit differences that matter to users.
   - Two songs may share the same genre but differ in mood, melody, and lyrical themes.
   - If only genre is represented, the system cannot capture these differences.

Limitations of Content-Based Recommendation

3. Overspecialization
   - Recommendations become too similar to previously liked items
   - Low diversity and limited exploration
   - May fail to discover interests not reflected in the user’s history.
4. User Cold Start
   - New users have limited interaction history
   - Difficult to build accurate user profiles initially
   - Item cold start is less severe when useful content features are available.

Collaborative Filtering Recommendation

Predict user preferences from patterns in user–item interactions.

- Memory-based
- Model-based

別人也看了



ASUS 華碩 Zenbook A14 14吋...

$41,999

~~$49,999~~



ASUS 華碩 Zenbook S 14 OL...

$55,999

~~$59,900~~



ASUS 華碩 Zenbook Duo 14...

$64,999

~~$68,999~~



ASUS 華碩 ExpertBook P5 14...

$45,900

~~$48,900~~



ASUS 華碩 Zenbook A14 14吋...

$36,999

~~$43,999~~



ASUS 華碩 Zenbook 14 OLE...

$44,999

~~$48,999~~



ASUS ROG 華碩 V16 16吋效...

$41,999



ASUS 華碩 Zenbook A14 14吋...

$41,999



ASUS 華碩 Zenbook A14 14吋...

$41,999



ASUS 華碩 Vivobook S14 14吋...

$35,999



ASUS 華碩 Zenbook A14 14吋...

$59,999



ASUS 華碩 Vivobook 16 16吋A...

$29,999

Collaborative Filtering

- Main idea
  - Predict user preferences by learning patterns from user–item interactions.
  - Use ratings, clicks, purchases, or other feedback across users.
  - Item content features are not required in pure CF.
- Goal: Predict missing ratings or rank candidate items for each user.

Collaborative Filtering Approaches

- Memory-based CF
  - Directly use observed interactions to compute similarities and make predictions.
  - User-based: Predict preferences using feedback from users with similar interaction patterns.
  - Item-based: Use similarities between items' interaction patterns.
- Model-based CF
  - Learn a predictive model from observed user–item interactions.
  - Example: Matrix factorization.

Memory-based Collaborative Filtering

User-based CF

○: Like ×: Dislike Blank: Unknown ?: Target prediction

	風聲	矮人礦坑	三國殺	富饒之城	地產大亨	西洋棋
Alice	O	X	O	?	X	X
Bob	O	O		O	X	X
Johnny	X	O	X	O		
Jack				X		O
Amy	O	X	O	O	X	X
Cathy	O		O		O	
David	X		X		O	

Alice and Amy have matching preferences on their co-rated items. Amy likes 富饒之城, so we predict that Alice may also like it.

Item-based CF

○: Like　×: Dislike　Blank: Unknown　?: Target prediction

	風聲	矮人礦坑	三國殺	富饒之城	地產大亨	西洋棋
Alice	O	X	O		X	X
Bob	O	O	?	O	X	X
Johnny	X	O	X	O		
Jack				X		O
Amy	O	X	O	O	X	X
Cathy	O		O		O	
David	X		X		O	

風聲 and 三國殺 have matching feedback patterns among users who rated both. Since Bob likes 風聲, we predict that he may also like 三國殺.

Memory-based CF

- Neighborhood-based Methods
  - Use similarities between users or items derived from the observed interaction matrix.
  - Generate predictions by aggregating feedback from relevant neighbors.
  - Often simple and interpretable.
  - Basic methods do not fit a separate predictive model, but may precompute similarities and neighborhoods.

Challenge in Memory-based CF

- Challenge: handling differences in rating scales
  - Users may use rating scales differently
  - Some users tend to give higher ratings than others.
- Mean-centered prediction
  - Compare ratings relative to each user’s average.
  - Use an adjusted weighted sum of neighbors’ rating deviations, then add the target user’s mean rating.
- Alternative: Relative preferences
  - Model which items a user prefers over others.

User	Average rating	Rating for an item	Deviation
A	4.5	4.0	−0.5
B	2.0	3.0	+1.0

User-based nearest-neighbor CF

- The basic technique:
  ① Choose a target user, Alice, and a candidate item i she has not rated.
  ② Select users with similar rating patterns who have rated item i.
  ③ Aggregate their ratings to predict Alice's rating for item i.
  ④ Repeat for candidate items and recommend those with the highest predicted ratings.

User-based nearest-neighbor CF

- Example: Predict Alice's Rating for Item5
  - The user–item rating matrix is shown below (ratings: 1–5).
  - Predict Alice's rating for Item 5 using other users' ratings.

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

User-based nearest-neighbor CF

- Key Design Questions
  a. How should similarity be measured?
  b. How many neighbors should be selected?
  c. How should neighbors' ratings be aggregated?
  d. Should users with negative similarity be included as neighbors?

Measuring user similarity

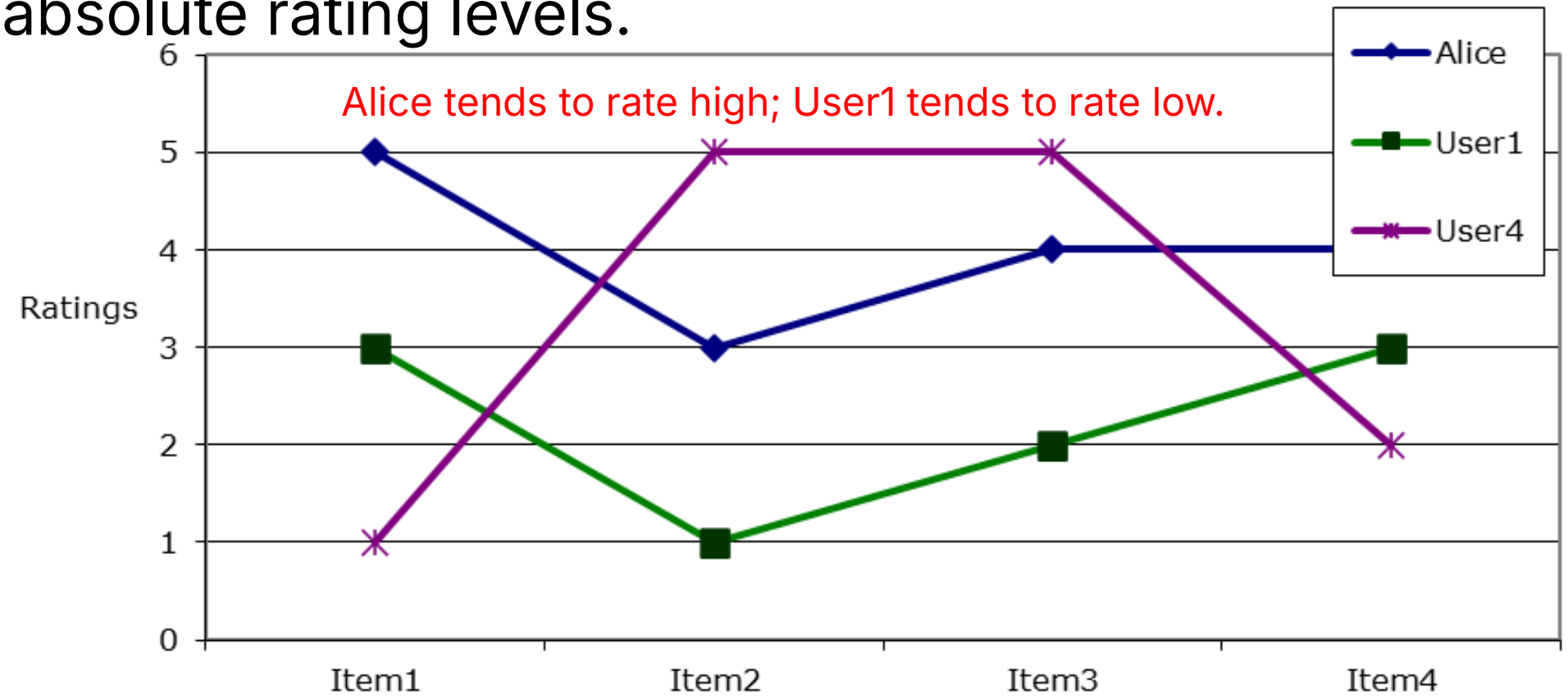
- A popular similarity measure in user-based CF: Pearson correlation

$$sim(a,b) = \frac{\sum_{p \in P}\left(r_{a,p} - \bar{r}_a\right)\left(r_{b,p} - \bar{r}_b\right)}{\sqrt{\sum_{p \in P}\left(r_{a,p} - \bar{r}_a\right)^2}\sqrt{\sum_{p \in P}\left(r_{b,p} - \bar{r}_b\right)^2}}, \bar{r}_a = \frac{1}{|P|}\sum_{p \in P} r_{a,p}$$

  - a, b: users
  - P: set of items rated both by a and b
  - $r_{a,p}$: rating of user a for item p
  - Both $\bar{r}_a$ and $\bar{r}_a$ are computed over the co-rated item set P

Pearson correlation

- Compares mean-centered rating patterns rather than absolute rating levels.



Measuring user similarity

- A popular similarity measure in user-based CF: Pearson correlation

$$sim(a,b) = \frac{\sum_{p \in P}(r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{\sum_{p \in P}(r_{a,p} - \bar{r}_a)^2}\sqrt{\sum_{p \in P}(r_{b,p} - \bar{r}_b)^2}}$$

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1



sim = 0.85
sim = 0.71
sim = 0.00
sim = -0.79

Select $k = 2$ neighbors with the highest positive similarities

Making predictions

- A common prediction function:

$$\widehat{r_{a,p}} = \bar{r}_a + \frac{\sum_{b \in N_p(a)} sim(a,b) * (r_{b,p} - \bar{r}_b)}{\sum_{b \in N_p(a)} |sim(a,b)|}$$

  - $N_p(a)$: selected neighbors of user a who rated item p.
  - Compute each neighbor's deviation from their average rating.
  - Aggregate these deviations using similarity weights.
  - Add the weighted deviation to the target user's average rating.

Pearson similarity uses means over co-rated genres.
Rating prediction uses each user's mean over all observed ratings.

	Pop	Rock	R&B	Jazz	Classical	Reggae
Alice	1	5		2	4	?
User1	4	2		5	1	2
User2	2	4	3			5
User3	2	4		5	1	

$$\bar{r}_{Alice} = \frac{1+5+2+4}{4} = 3$$

$$\bar{r}_{User1} = \frac{4+2+5+1+2}{5} = 2.8$$

$$\bar{r}_{User2} = \frac{2+4+3+5}{4} = 3.5$$

$$sim(Alice, User1) = \frac{(1-3)*(4-3)+(5-3)*(2-3)+(2-3)*(5-3)+(4-3)*(1-3)}{\sqrt{(1-3)^2+(5-3)^2+(2-3)^2+(4-3)^2}\sqrt{(4-3)^2+(2-3)^2+(5-3)^2+(1-3)^2}} = -0.8$$

$$sim(Alice, User2) = \frac{(1-3)*(2-3)+(5-3)*(4-3)}{\sqrt{(1-3)^2+(5-3)^2}\sqrt{(2-3)^2+(4-3)^2}} = 1$$

$$pred(Alice, Reggae) = 3 + \frac{-0.8*(2-2.8)+1*(5-3.5))}{0.8+1} = 4.19$$

Both positive and negative similarities are used in this example.

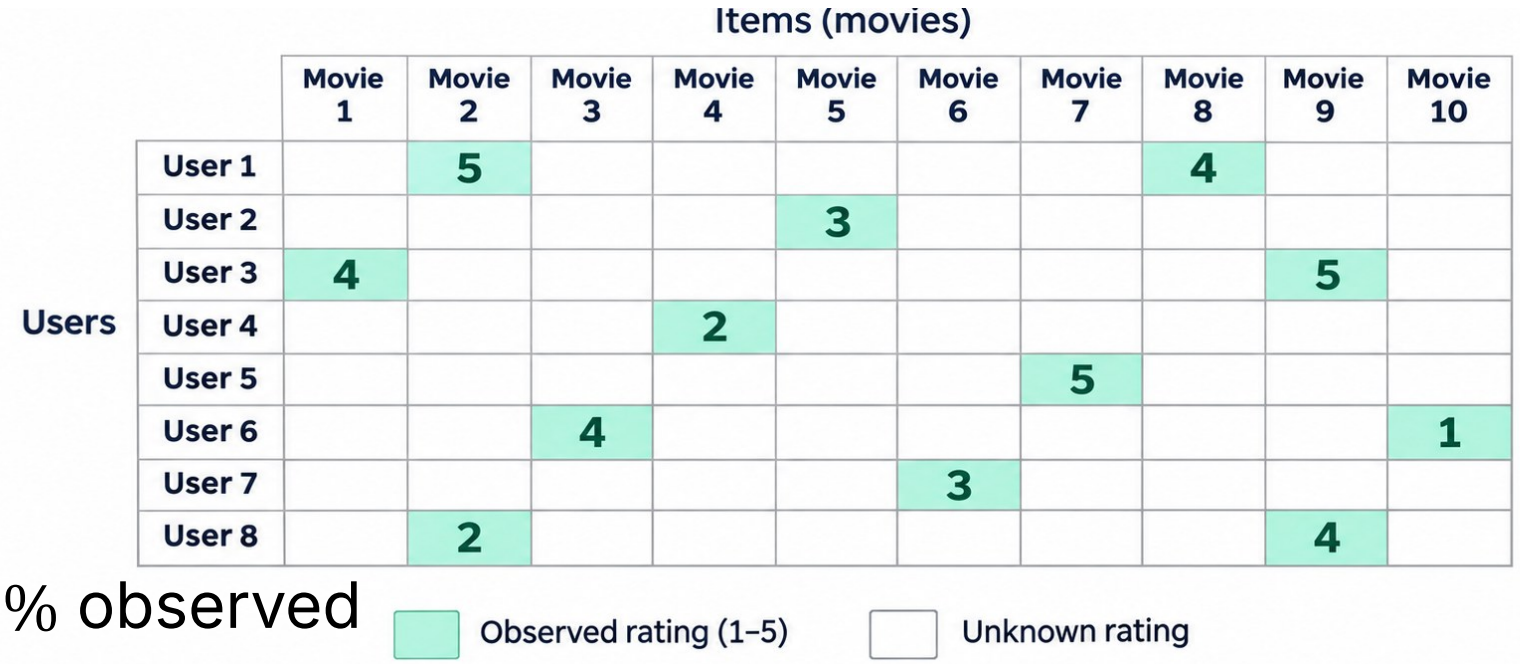
Model-based Collaborative Filtering

Central problem of recommendation

- The number of observed interactions is extremely sparse compared with all possible user-item pairs
  - Most user-item preferences are unknown
  - The rating matrix is extremely sparse
- Recommendation systems can be viewed as missing-value prediction problems
- Matrix Completion: $R \approx UV^T$
  - Approximate the sparse rating matrix using low-dimensional latent representations
  - $U \in \mathbb{R}^{m*k}$: user latent matrix
  - $V \in \mathbb{R}^{n*k}$: item latent matrix

Central Problem of Recommendation

- Only a small fraction of all possible user–item interactions are observed.
  - Most user-item preferences are unknown
  - The rating matrix is extremely sparse



$\frac{12}{80} = 15\%$ observed

Central Problem of Recommendation

- Rating prediction can be formulated as predicting missing entries in a partially observed matrix.
- Matrix completion via low-rank matrix factorization: $\hat{R} = UV^T$
  - Learn low-dimensional user and item factors from observed ratings, then predict missing entries.
    - $U \in \mathbb{R}^{m \times k}$: user factor matrix
    - $V \in \mathbb{R}^{n \times k}$: item factor matrix
    - m: number of users
    - n: number of items
    - k: number of latent factors

Model-based Collaborative Filtering

	風聲	矮人礦坑	三國殺	富饒之城	地產大亨	西洋棋
Alice	O	X	O		X	X
Bob	O	O		O	X	X
Johnny	X	O	X	O		
Jack	?			X		O
Amy	O	X	O	O	X	X
Cathy	O		O		O	
David	X		X		O	

Model-based Collaborative Filtering

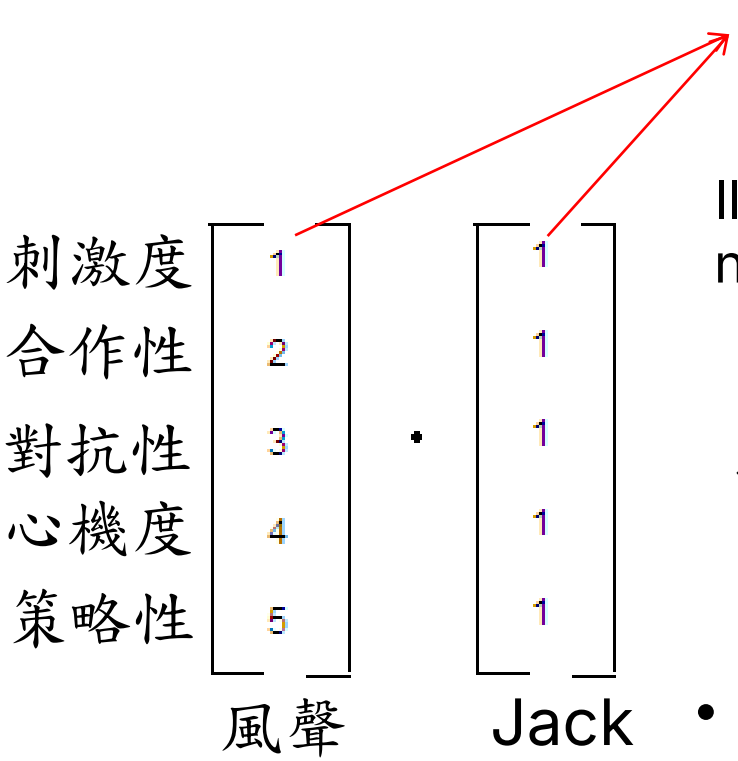


Illustration only: Learned latent dimensions do not necessarily correspond to named game attributes

$$v_{\text{風聲}}^{T} u_{Jack} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = 15$$

- Latent vectors are not manually defined
- The system learns them automatically from data

Matrix Factorization for CF

- Learn low-dimensional user and item vectors from observed interactions.
- Predict ratings using the dot product of these vectors.
  - Recommend items with high matching scores
- Standard singular value decomposition (SVD) does not directly handle missing entries.
- Recommendation models can instead optimize prediction error only on observed entries.

Learning the Latent Factors

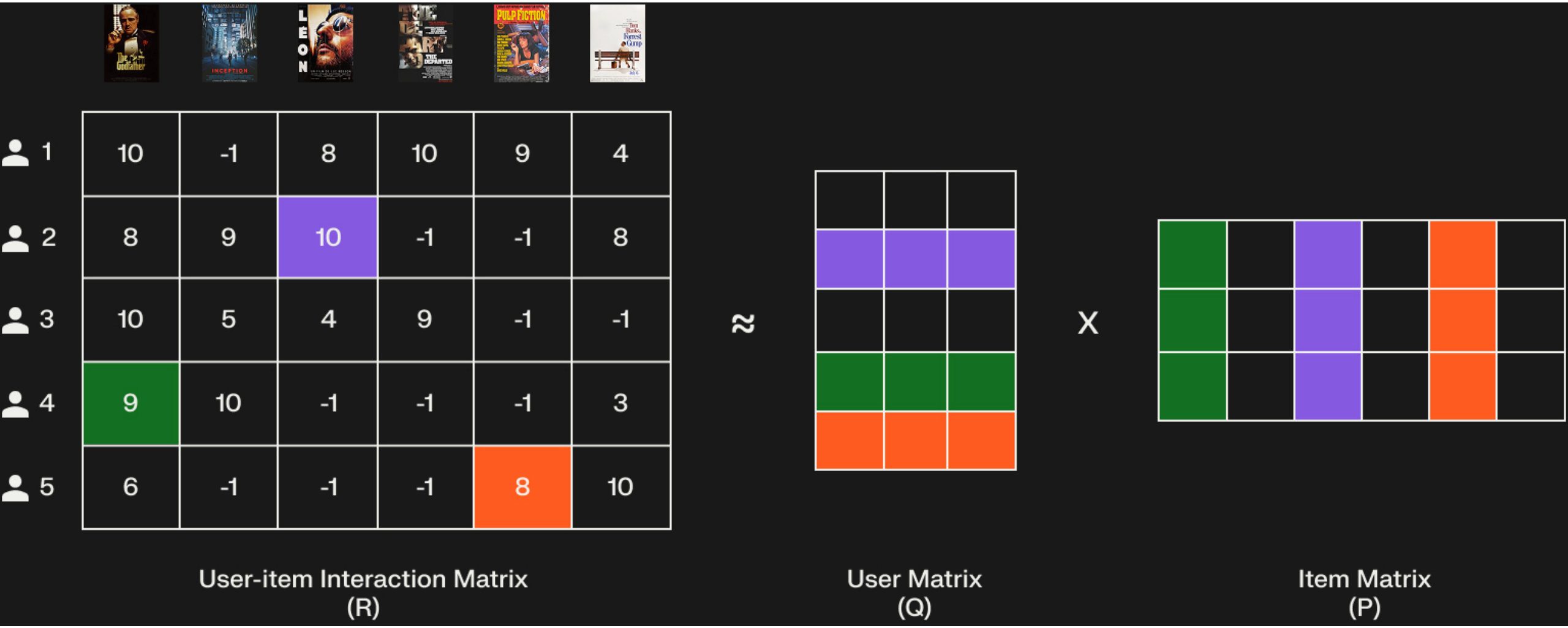
- For all $r_{u,i}$ known in the user-interaction matrix

$$\min_{U,V} \sum_{(c,s)\in\Omega} (r_{cs} - u_c^T v_s)^2 + \lambda(\|U\|_F^2 + \|V\|_F^2)$$

only observed ratings

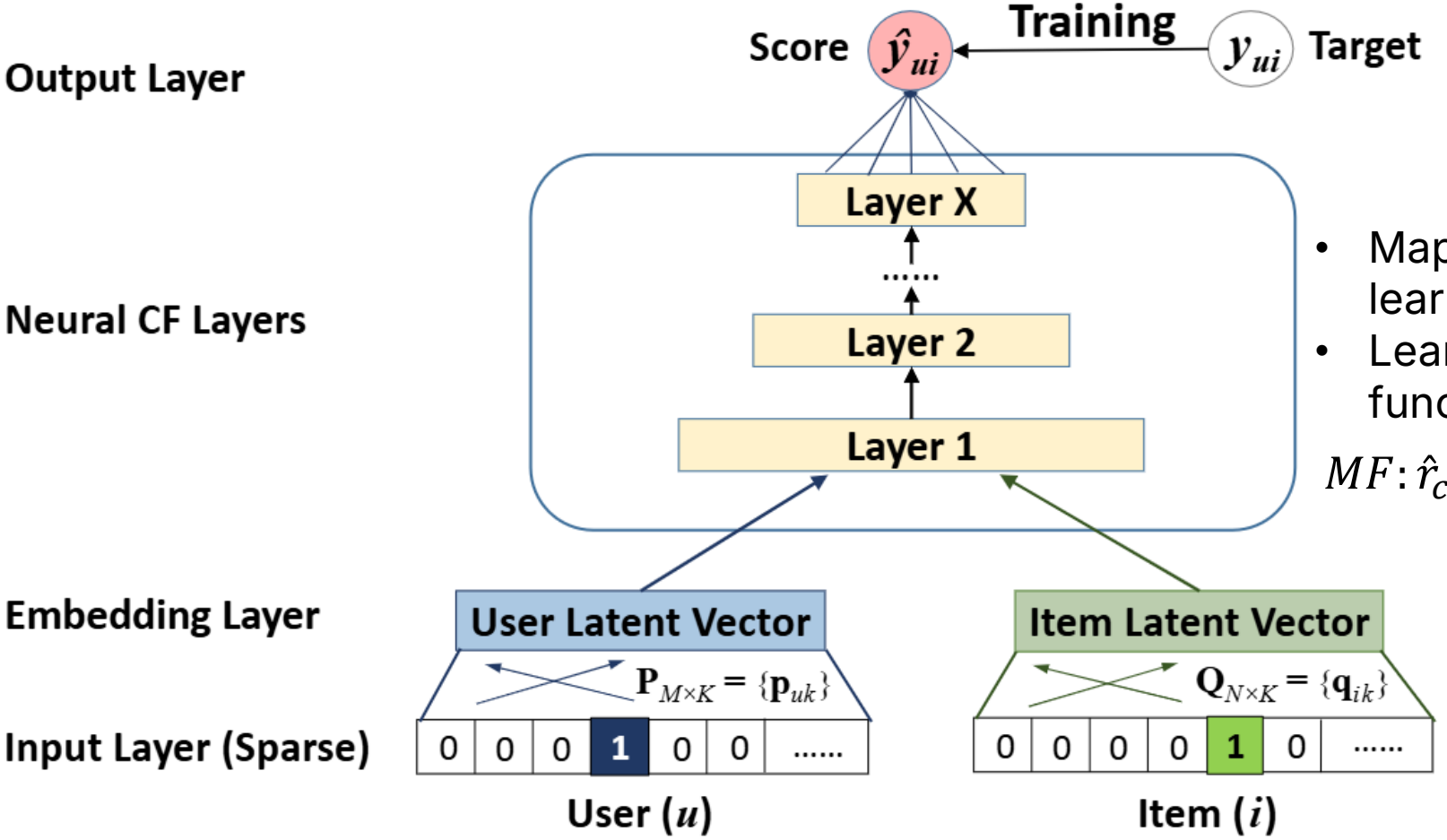
Regularization penalizes large factor values and helps reduce overfitting.

  - Approximate the rating matrix by: $\hat{R} = UV^T$
    - U is user latent matrix and V is item latent matrix
  - Predicted Rating: $\hat{r}_{cs} = u_c^T v_s$
    - $u_c$is user latent vector and v_s is item latent vector
  - Learn user/item embeddings that minimize rating prediction error.
    - Users and items are projected into the same latent space
    - Users or items with similar interaction patterns may learn similar latent representations.
    - Optimize the factors using stochastic gradient descent or alternating least squares.



−1 denotes missing ratings and is excluded from the training loss

Neural Collaborative Filtering (NCF)

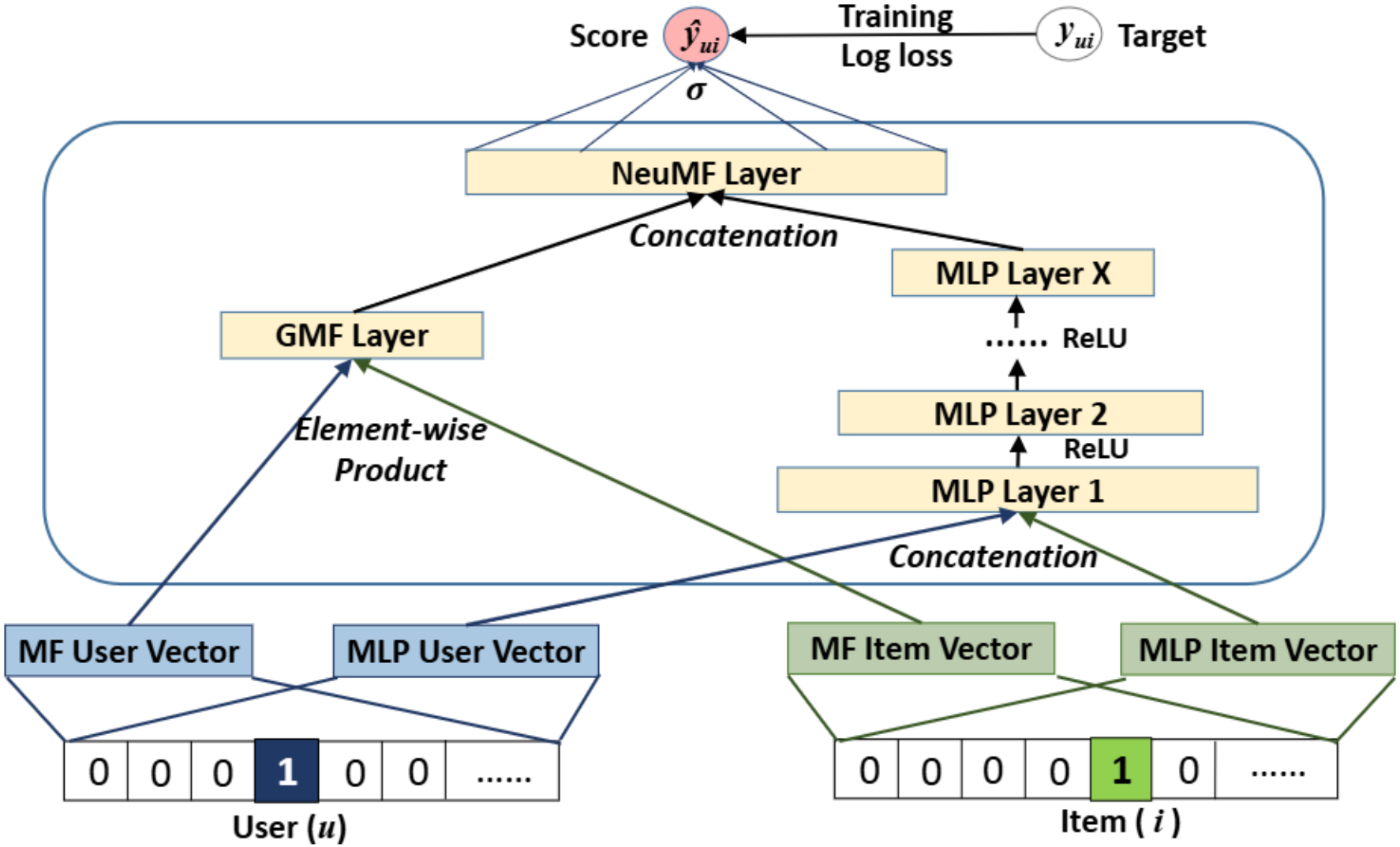


- Map user and item IDs to learned embeddings.
- Learn the user–item interaction function using neural networks.

$$MF: \hat{r}_{cs} = u_c^T v_s \Rightarrow NCF: f_\theta(c, s)$$

Neural Collaborative Filtering, 2017, WWW

NeuMF: Combining GMF and MLP



Recommender systems

Cold-Start Problem

Cold-Start Problem

- Collaborative filtering learns user preferences from historical interactions.
- Cold-start problems occur when:
  - New users have few or no recorded interactions.
  - New items have few or no recorded interactions.
- With little or no interaction data, pure CF struggles to estimate reliable similarities or latent vectors.

Types of Cold-Start

- New User Problem
  - A user is new to the system or target domain.
  - Few or no recorded interactions are available in the target system or domain.
- New Item Problem
  - A new movie/product/song is added.
  - Few or no users have interacted with it yet.
- System Cold-Start
  - Entire platform is new.
  - Little or no historical interaction data is available.

Solutions to Cold-Start

① Elicit Initial Preferences
- Ask new users to select favorite items, artists, or categories.

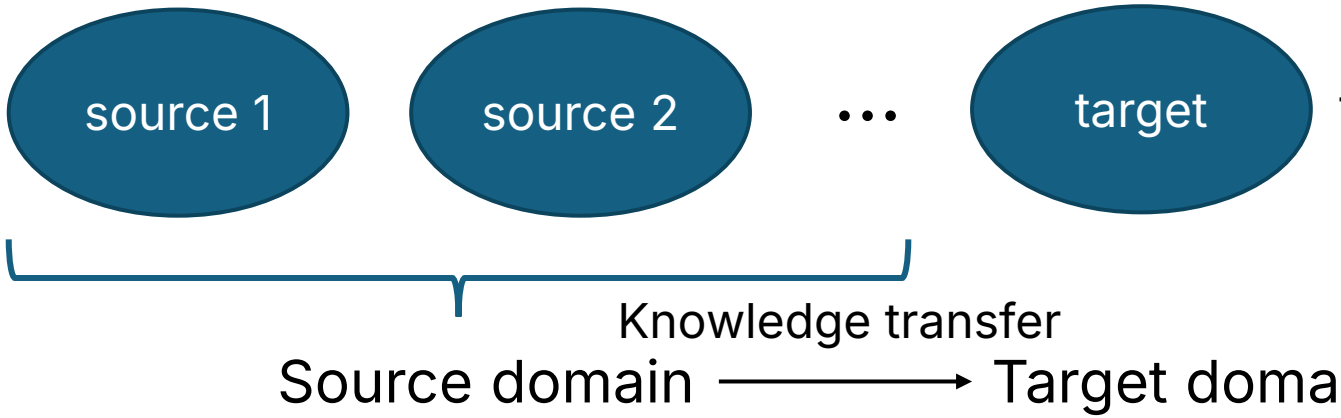
② Content-based and Hybrid Recommendation
- Use item features and user profiles, optionally combined with collaborative signals.

③ Popularity-based Recommendation
- Offer popular items as an initial fallback for new users.

④ Cross-domain Recommendation
- Transfer useful preference information from another domain.

Cross-domain Recommendation

- Domains can differ in item categories, content types, or platforms.
  - Item category: Thriller movies and comedy movies.
  - Content type: Movies and TV shows.
  - Product category: Movies and books.
  - Platform: Disney+ and Netflix.

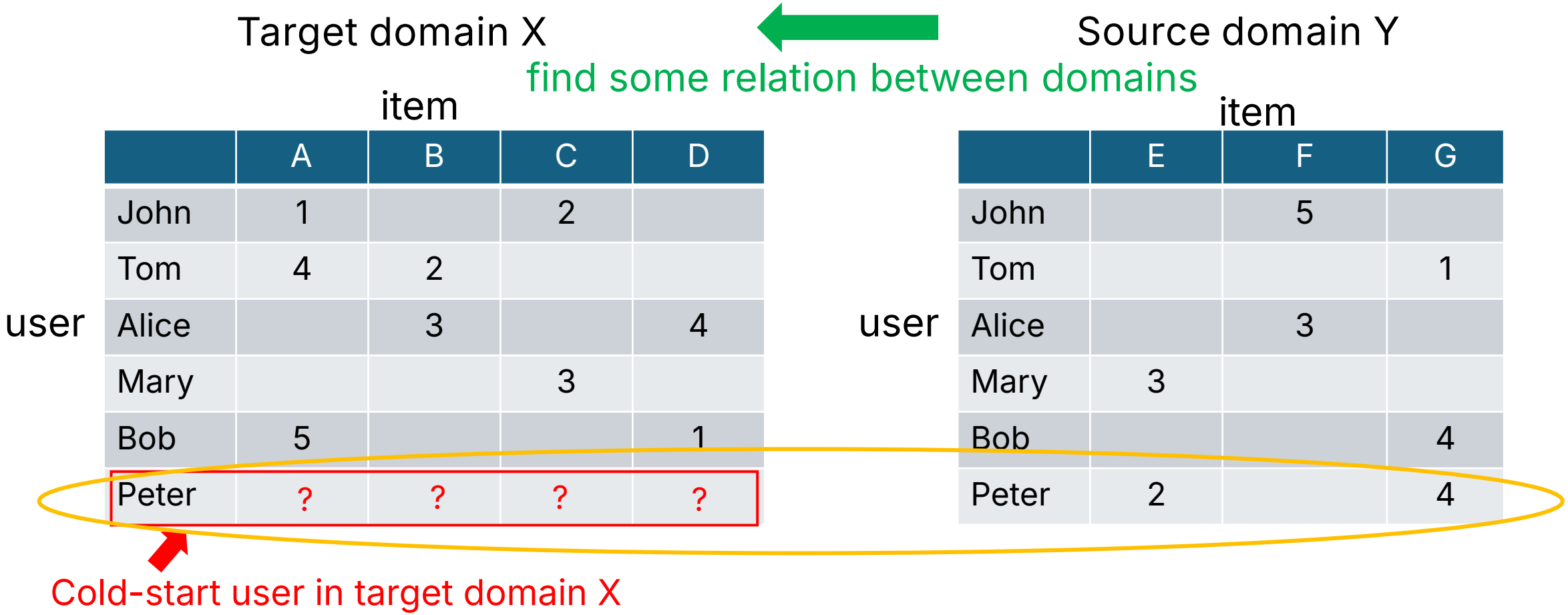


Source domain: provides knowledge for transfer.
Target domain: where recommendations are to be improved.

Cross-domain Recommendation

- A user is new to the movie domain:
  - No movie interactions are available.
  - Many music-listening interactions are available.
- Can preferences learned from music help recommend movies?
  - One Approach: Shared Representation Learning
    - Different domains may have different feature spaces.
    - Learn aligned latent representations that capture transferable preferences.
    - Use shared users or other cross-domain links to connect the domains.

Transferring Preferences Across Domains



	A	B	C	D
John	1		2	
Tom	4	2		
Alice		3		4
Mary			3	
Bob	5			1
Peter	?	?	?	?

	E	F	G
John		5	
Tom			1
Alice		3	
Mary	3		
Bob			4
Peter	2		4

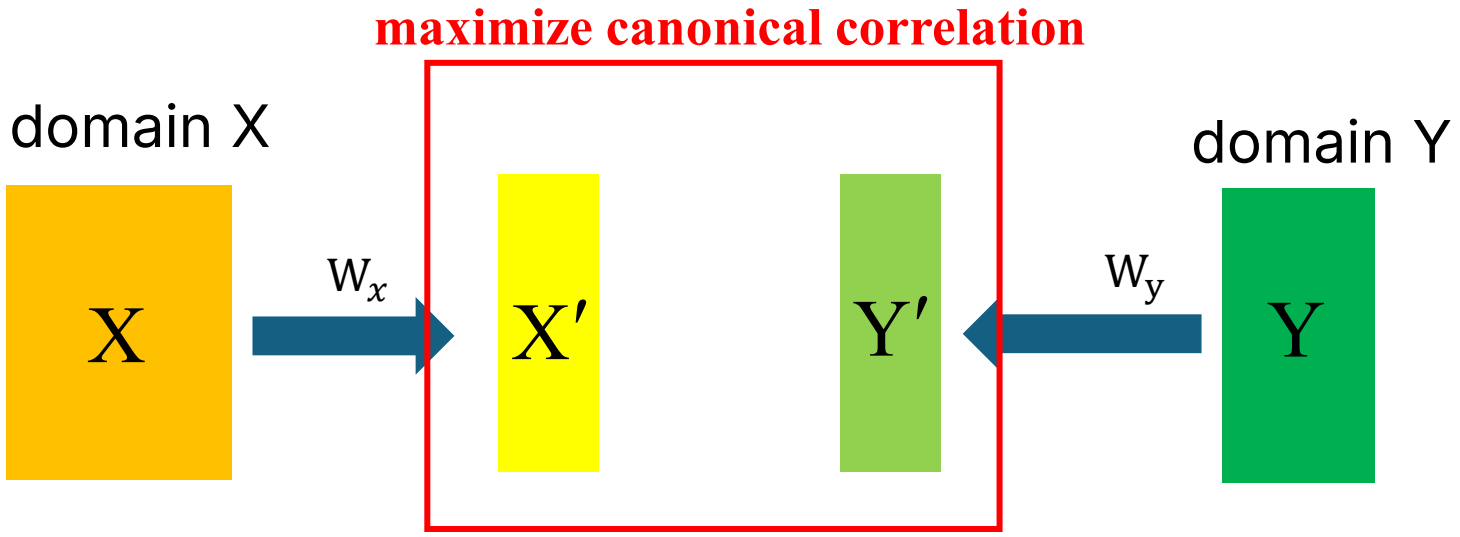
How to Learn a Shared Latent Space

- One approach: jointly factorize interaction matrices with shared user factors.
  - The same user is represented by a shared latent vector across domains.
  - Suppose we have music domain: X_{music} and movie domain: X_{movie}
    - Use a shared user factor matrix U for users appearing in both domains.
    - Learn separate item factor matrices for music and movies.
    - Train jointly on observed interactions from both domains.

$$\hat{R}^{music} = U\left(V^{music}\right)^T, \hat{R}^{movie} = U\left(V^{movie}\right)^T$$

Statistical Correlation based Method: Canonical Correlation Analysis (CCA)

- CCA learns linear projections of two paired data views that maximize their correlation.
- It can align representations from different domains or modalities.



Canonical Correlation Analysis

- Given two centered data matrices with n paired samples:
  - $X \in R^{n \times d_x}, Y \in R^{n \times d_y}$
- Learn projection vectors:
  - $w_{\mathrm{x}} \in R^{d_x}, \mathrm{w_y} \in R^{d_y}$
- Maximize the correlation between projected values:

$$\max_{w_x, w_y} corr(Xw_x, Yw_y)$$

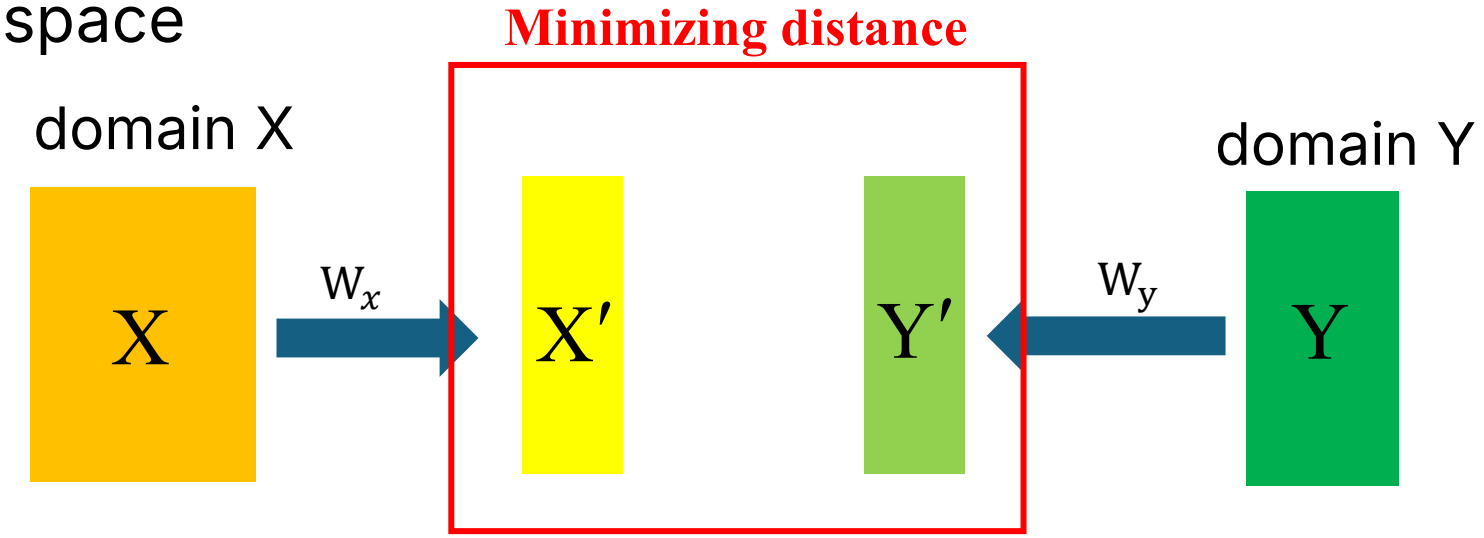
  - Capture linearly correlated patterns across the two views
  - Corresponding rows represent the same user in the two domains.

Distance-based Shared Space Learning: Cross-modal Factor Analysis (CFA)

- Learn a shared latent space while minimizing cross-modal distance

$$\min_{W_x, W_y} \left\| XW_x - YW_y \right\|_F^2, W_x^T W_x = I, W_y^T W_y = I$$

  - Paired samples from different domains should become close in latent space



Multimedia content processing through cross-modal association, ACM MM, 2003

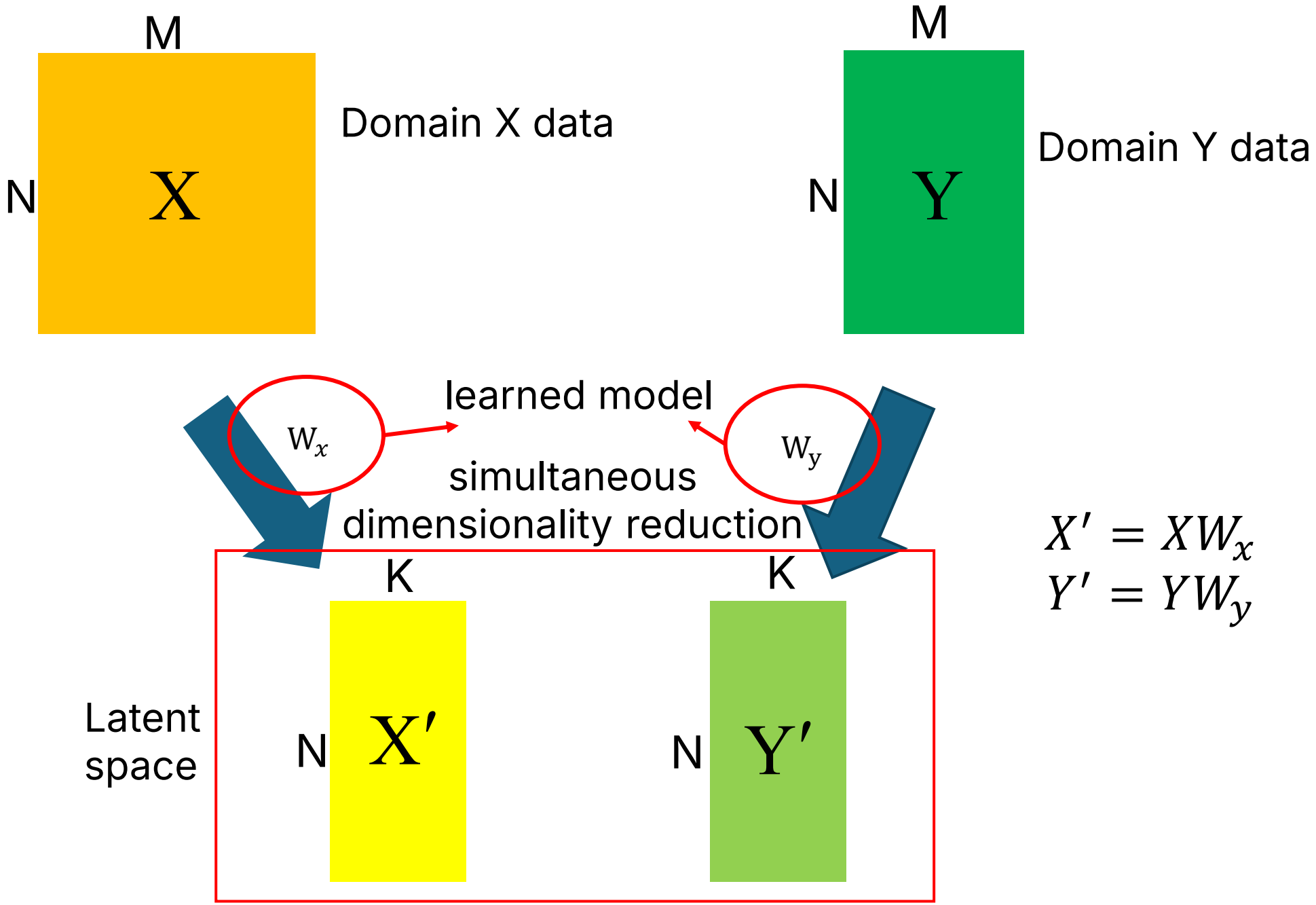
Regression-based Rating Prediction: Partial Least Squares Regression (PLSR)

- Partial Least Squares Regression (PLSR) reduces dimensionality and uses latent components to predict target outputs.
  - For the first pair of latent components:

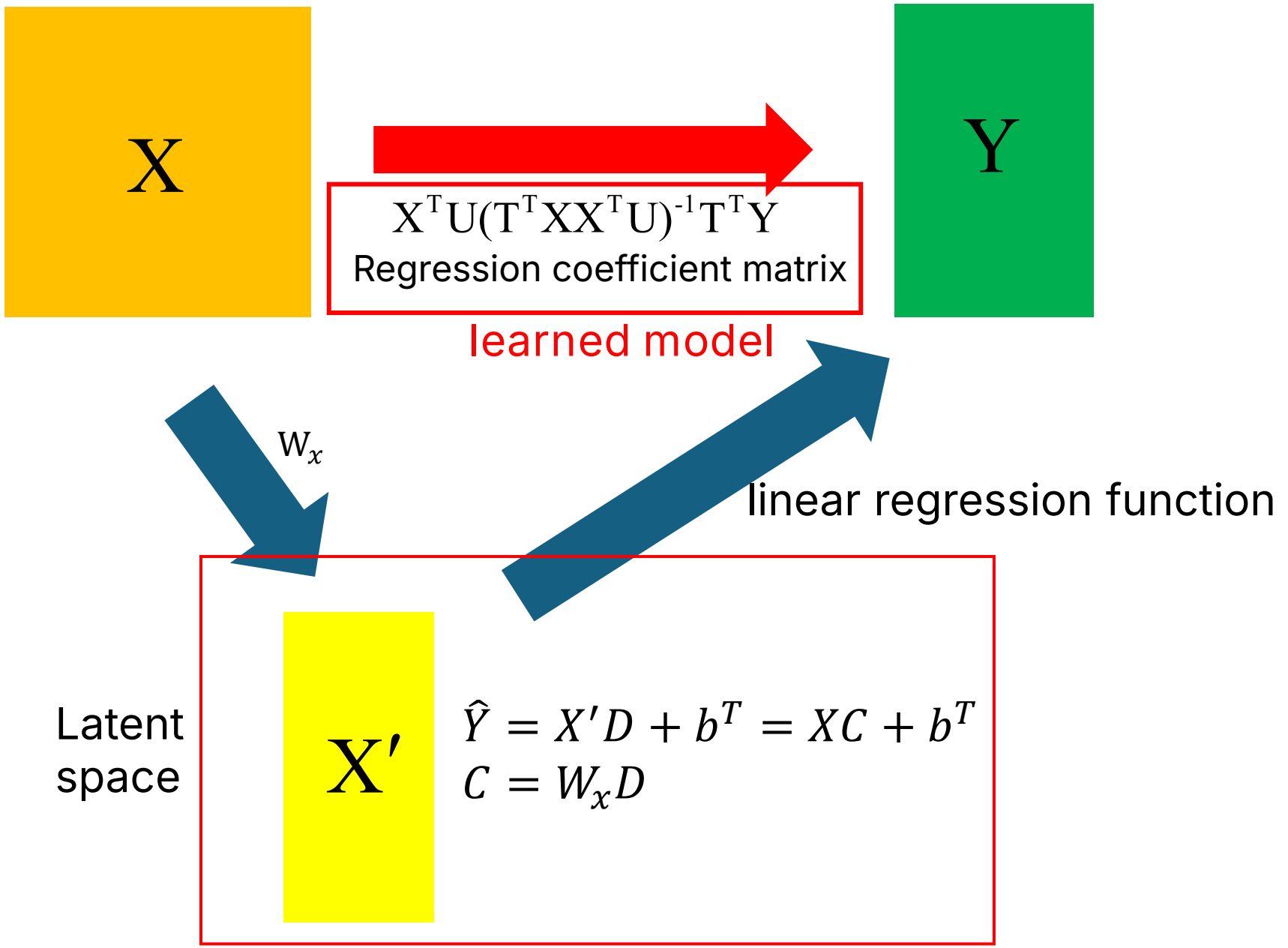
$$\max_{w_x, w_y} cov(Xw_x, Yw_y)\ s.t\ \|w_x\|_2 = \|w_y\|_2 = 1$$

  - PLSR learns latent components from predictors X and responses Y.
  - It selects components with high cross-covariance and uses them to predict Y from X.

A cross-domain recommendation mechanism for cold-start users based on partial least squares regression, 2018, ACM TIST


M
N
X
Domain X data
M
N
Y
Domain Y data
learned model
W_x
W_y
simultaneous
dimensionality reduction
K
K
Latent
space
N
X′
N
Y′

$$X' = XW_x$$
$$Y' = YW_y$$


X
Y
$X^T U(T^T X X^T U)^{-1} T^T Y$
Regression coefficient matrix
learned model
W_x
linear regression function
Latent space
X′
$\hat{Y} = X'D + b^T = XC + b^T$
$C = W_x D$

Extensions to Cross-domain Recommendation

- Nonlinear representation alignment
  - Kernel CCA
  - Deep CCA
- Contrastive representation learning
  - Align related users or items across domains (e.g., triplet loss)
  - CLIP-style embedding alignment
- Foundation-model-assisted recommendation
  - Use pretrained representations of item descriptions, images, or audio
  - Use LLMs to extract transferable preferences from user histories.

Evaluating Recommender Systems

- Rating Prediction
  - Predict a user's numerical rating for an item.
  - Evaluate errors on held-out observed ratings.
    - MAE: Average absolute prediction error.
    - RMSE: Gives larger errors a greater penalty.
    - Lower values indicate better predictions.
- Top-K Recommendation
  - Recommend a ranked list of K items.
    - Precision@K: What fraction of recommended items are relevant?
    - Recall@K: What fraction of relevant items are retrieved?
    - NDCG@K: Are highly relevant items ranked near the top?
    - Higher values indicate better recommendations.

Example: Top-K Evaluation

- Relevant items for a user in the test set $G_u = \{A, C, F\}$
- The system recommends: $L_u = [A, B, C], K = 3$
  - $Precision@3 = \frac{|L_u \cap G_u|}{3} = \frac{2}{3}$
  - $Recall@3 = \frac{|L_u \cap G_u|}{|Gu|} = \frac{2}{3}$
  - $NDCG@3 = \frac{DCG@3}{IDCG@3} = 0.704$
    - $DCG@3 = \sum_{i=1}^{3} \frac{2^{rel_i} - 1}{\log_2(i+1)} = \frac{1}{\log_2(2)} + \frac{0}{\log_2(3)} + \frac{1}{\log_2(4)}$
    - $IDCG@3 = \sum_{i=1}^{3} \frac{2^{rel_i} - 1}{\log_2(i+1)} = \frac{1}{\log_2(2)} + \frac{1}{\log_2(3)} + \frac{1}{\log_2(4)}$

rel_i: relevance scores

Summary

- Recommendation Tasks
  - Predict ratings or rank candidate items for each user.
- Content-Based Recommendation
  - Match item features to a user’s preference profile.
- Collaborative Filtering
  - Neighborhood methods aggregate feedback from similar users or items.
  - Matrix factorization and neural models learn from user–item interactions.
- Cold Start
  - Limited interactions make personalization difficult.